

Human Action Recognition with a from-scratch C++23 Video CNN

AMSC Project Report

AMSC-HAR

Politecnico di Milano

Advanced Methods for Scientific Computing

August 30, 2026

Abstract

This report describes **AMSC-HAR**, a header-only C++23 library and command-line trainer for video-based human action recognition. Unlike typical student projects that wrap PyTorch or TensorFlow, the system implements tensors, layers, automatic-style reverse-mode gradients, SGD, checkpointing, video decoding, and optional CUDA kernels from scratch. The model is a compact VideoCNN: a shared 2D convolutional backbone over sampled RGB frames, a residual temporal 1D convolution, learnable temporal attention pooling, and a linear classifier. On the UCF11 (YouTube Action) dataset the default recipe uses 64×64 frames, $T = 8$ uniformly sampled time steps, and about 5.9×10^5 trainable parameters. A held-out inference check with the released best checkpoint `ucf11_videocnnBest.harw` classified 8/10 randomly drawn clips correctly (80%), with several classes predicted at $> 95\%$ confidence. The same codebase also targets UCF101 official splits. This document covers design goals, software architecture, the mathematical model, the data pipeline, training, the CUDA backend, and experimental results.

Contents

1	Introduction	2
2	Background and problem statement	3
2.1	Video action recognition	3
2.2	Datasets	4
2.3	Design constraints	4
3	Software architecture	4
3.1	Build system	4
3.2	Command-line interface	4
3.3	Checkpoint format	5

4	Tensor library and layers	5
4.1	Tensors	5
4.2	Layer contract	5
4.3	Spatial operators	6
4.4	Optimiser	6
5	VideoCNN architecture	6
5.1	Forward pass	6
5.2	Temporal convolution and attention	7
5.3	Capacity	7
6	Data pipeline	8
6.1	Decoding	8
6.2	Cache and model clip	8
6.3	Prefetch	8
7	Training procedure	8
8	CUDA backend	9
9	Experiments	9
9.1	Setup	9
9.2	Folder test on 10 random UCF11 clips	10
9.3	How to read the best checkpoint	10
9.4	Smoke test and software correctness	10
9.5	Limitations	10
10	Conclusions	10

1 Introduction

Human action recognition (HAR) from video represents a significant area of research within the domain of computer vision, addressing challenges such as real-world variability, computational efficiency, and robust feature extraction [1]. Recent literature highlights the synergistic combination of Soft Computing (SC) paradigms—such as Neural Networks—and Multi-Agent Systems (MAS) as a robust strategy to overcome these challenges [1]. A short clip must be mapped to one of a fixed set of action classes (e.g. *diving*, *tennis swing*, *walking a dog*). The academic community usually solves this with large 3D CNNs or transformer video models trained in Python frameworks. Those stacks hide the linear algebra, memory layout, and gradient bookkeeping that a scientific-computing course is meant to expose.

Following the directions outlined by Houssein et al. [1] for efficient and robust HAR systems, AMSC-HAR takes the opposite route to traditional frameworks. The project represents a foundational Soft Computing component (a from-scratch Neural Network) capable of being integrated into larger distributed Multi-Agent frameworks, providing a *complete* recognition pipeline written in modern C++:

- a generic `Tensor<T>` with CPU storage and optional CUDA Unified Memory;
- layers with explicit `forward/backward` (Conv2D, BatchNorm2D, ReLU, pooling, Dropout, Linear, TemporalConv1D, temporal attention);
- SGD with momentum and ℓ_2 weight decay;
- softmax cross-entropy;
- OpenCV (or FFmpeg) video decoding, a binary clip cache, and train-time augmentation;
- a custom checkpoint format `.harw`;
- a single executable `har_cnn` for smoke tests, training, prediction, and split evaluation.

Contributions. (1) A compact, auditable VideoCNN that stays under 0.6M parameters while remaining strong on UCF11. (2) A dual CPU/CUDA path so the same headers train on a laptop or a GPU. (3) An end-to-end data path (decode $\rightarrow$ cache $\rightarrow$ temporal sample $\rightarrow$ batch) that makes repeated epochs practical. (4) Reproducible CLI defaults (seed 42, 80/10/10 UCF11 split) and a self-describing weight file that stores image size, clip length, and class names.

Report outline. Section 2 recalls HAR and the datasets. Section 4 describes tensors and layers. Section 5 specifies VideoCNN. Section 6 covers decoding and augmentation. Section 7 details the training loop. Section 8 summarises CUDA kernels. Section 9 reports experiments. Section 10 concludes.

2 Background and problem statement

2.1 Video action recognition

Action recognition represents a vital domain within computer vision, utilizing various data types to analyze human activities [1]. A video is a tensor $X \in \mathbb{R}^{T \times C \times H \times W}$. The classifier produces logits $z \in \mathbb{R}^K$ and a predicted class $\hat{y} = \arg \max_k z_k$. As highlighted by recent studies [1], traditional methods are being replaced by Deep Learning and Soft Computing techniques capable of automatically extracting spatio-temporal features. While recent work uses 3D CNNs and space-time transformers which are accurate but computationally expensive, efficient models are required for real-time deployment and distributed sensing environments.

AMSC-HAR uses a cheaper and more transparent design: *2D CNN on each frame*, then a *1D temporal mixer* and *attention pooling*. This late temporal aggregation provides a transparent Soft Computing design where the temporal conv and attention are trained jointly with the backbone, ensuring computational efficiency [1].

2.2 Datasets

UCF11 (YouTube Action) contains 11 sport and daily actions, originally published as MPEG clips crawled from YouTube. This project uses the updated `UCF11_updated_mpg` release (1600 clips after scanning the extracted tree). UCF11 has no official train/test list, so the loader applies a class-stratified shuffle (seed 42) with default ratios 80% train / 10% val / 10% test.

UCF101 has 101 classes and three official recognition splits. The same VideoCNN is reused; only the classifier width and the split reader change.

Table ?? lists the UCF11 classes as stored in the best checkpoint (alphabetical folder names).

3 Tensor library and layers

3.1 Tensors

`har::Tensor<T>` is a dense, row-major, contiguous array with an explicit shape and computed strides. Images and activations use NCHW layout, which matches both the CPU convolution loops and the CUDA kernels. Allocation is either `new T[n]` on the host or `cudaMallocManaged` when the default device is CUDA. Elementwise arithmetic, `reshape/reshaped`, and `sync()` (a device-wide barrier when CUDA is active) are provided in the header.

A `Parameter<T>` pairs a data tensor with a same-shaped gradient and a `requires_grad` flag. SGD and checkpointing iterate parameter lists; BatchNorm additionally exposes non-parameter running mean/variance in the state dict.

3.2 Layer contract

Every spatial layer implements

$$y = f(x; \theta), \quad g_x = \frac{\partial L}{\partial x} = \left(\frac{\partial y}{\partial x} \right)^\top g_y,$$

by caching x (and any auxiliaries) during `forward` and using them in `backward`. There is no tape interpreter: the graph is the C++ call stack of `VideoCNN::forward / backward`. This is easy to debug and maps one-to-one onto the kernels in `cuda_ops.cu`.

3.3 Spatial operators

Conv2D. A $K_h \times K_w$ kernel with stride (s_h, s_w) and padding (p_h, p_w) implements

$$y_{n,c',h',w'} = b_{c'} + \sum_{c,i,j} W_{c',c,i,j} x_{n,c,s_h h' + i - p_h, s_w w' + j - p_w}.$$

Weights use a simple scaled-uniform init. The CPU path is a direct 7-loop implementation; the CUDA path launches `conv2d_forward/conv2d_backward`.

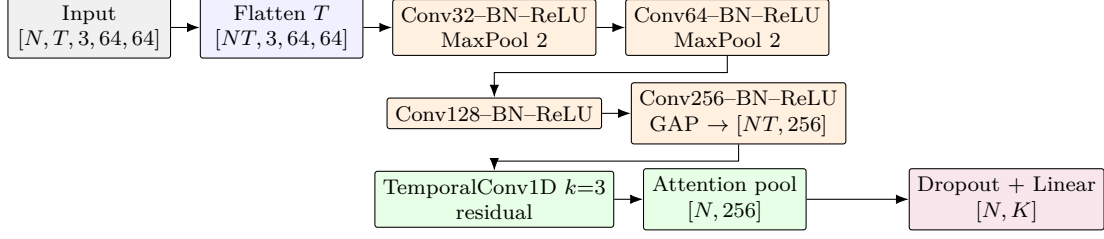


Figure 1: VideoCNN data path. Spatial stages are shared across time; the temporal block is residual ($h + \text{TConv}(h)$).

BatchNorm2D. Channel statistics are computed over $N \cdot H \cdot W$. In training,

$$\hat{x}_c = \frac{x_c - \mu_c}{\sqrt{\sigma_c^2 + \varepsilon}}, \quad y_c = \gamma_c \hat{x}_c + \beta_c,$$

and (μ, σ^2) are written into exponential moving averages (momentum 0.1). Eval uses the running statistics stored in the checkpoint, which is essential for the 80% folder test in Section 9.

Pooling and activations. 2×2 max-pool records argmax indices for the backward pass. Global average pool collapses $H \times W$ to a 256-D frame embedding. ReLU is $y = \max(x, 0)$. Dropout ($p = 0.3$) is applied only on the pooled vector, and only in training.

Linear and loss. The head is $z = Wh + b$ with $W \in \mathbb{R}^{K \times 256}$. Cross-entropy is the stable softmax + NLL

$$\ell = -\frac{1}{N} \sum_{n=1}^N \log \frac{e^{z_{n,y_n} - \max_k z_{n,k}}}{\sum_j e^{z_{n,j} - \max_k z_{n,k}}}.$$

The backward gradient is $(\text{softmax}(z) - e_y)/N$, moved back to the activation device.

3.4 Optimiser

SGD with momentum $m = 0.9$ and weight decay $\lambda = 10^{-4}$ updates

$$v \leftarrow mv + (g + \lambda\theta), \quad \theta \leftarrow \theta - \eta v.$$

The learning rate starts at $\eta = 3 \cdot 10^{-3}$ and is multiplied by $\gamma = 0.5$ every 4 epochs (`-lr-step`). Velocity buffers are allocated lazily to match parameter shapes and devices.

4 VideoCNN architecture

4.1 Forward pass

Let a batch of clips be $X \in \mathbb{R}^{N \times T \times 3 \times H \times W}$ with default $H = W = 64$ and $T = 8$. Time is flattened to a 2D batch of NT frames. A four-stage backbone produces a 256-D vector per frame; a temporal residual block mixes neighbouring frames; attention pools time; a linear layer outputs K logits (Figure 1).

Spatial spatial sizes after padding-1 3×3 convolutions: $64 \rightarrow 32$ (pool), $32 \rightarrow 16$ (pool), then 16×16 until GAP. Because image size must be divisible by 4, `build_video_cnn` rejects illegal `-size` values.

4.2 Temporal convolution and attention

TemporalConv1D treats the NT frame embeddings as N sequences of length T and applies a depth-preserving 1D convolution of kernel size 3 with zero-pad 1:

$$u_{n,t,c'} = b_{c'} + \sum_{c=1}^{256} \sum_{k=-1}^1 W_{c',c,k} h_{n,\text{clamp}(t+k),c}.$$

The residual $h + u$ lets the network keep a pure per-frame representation when the temporal kernel is not yet useful.

Temporal attention pooling scores each time step with a learned query $w \in \mathbb{R}^{256}$ (zero-initialised, so the first step equals a temporal mean):

$$\begin{aligned} s_{n,t} &= h_{n,t}^\top w, & \alpha_{n,t} &= \frac{e^{s_{n,t} - \max_{t'} s_{n,t'}}}{\sum_{t'} e^{s_{n,t'} - \max_{t''} s_{n,t''}}}, \\ \bar{h}_n &= \sum_{t=1}^T \alpha_{n,t} h_{n,t}. \end{aligned} \tag{1}$$

The classifier then sees a single 256-D clip embedding.

4.3 Capacity

Table ?? counts trainable tensors for $K = 11$. The last 3×3 convolution and the temporal kernel dominate. The released best file is 2 361 619 bytes and contains 29 tensors (parameters + four pairs of BN running stats), matching this layout exactly.

A smaller historical checkpoint (`ucf11_videocnn.64x8.harw`, ≈ 0.58 MB) corresponds to an earlier narrower backbone. A 112×16 variant exists for higher-resolution experiments, at a much larger activation cost.

5 Data pipeline

5.1 Decoding

`extract_video_frames` opens a file with OpenCV (`CAP_FFMPEG`, then Media Foundation, then `CAP_ANY`) or with FFMpeg. Frames are resized to $H \times W$, converted to RGB, scaled to $[0, 1]$, and stored as $[F, C, H, W]$ on the CPU. When `uniform=true` (the dataset default), F indices are spaced over the reported frame count so a long YouTube clip is summarised rather than truncated at the beginning.

UCF11 lives in a two-level tree `<class>/<group>/video.mpg`. The dataset walker skips annotation folders, sorts class names, and assigns integer labels in that order—the same order written into `.harw` class names.

```

Algorithm 1 Train VideoCNN on a HAR dataset.
Build model; optionally load_checkpoint(resume).
for epoch = 1 ...  $E$  do
  step-decay  $\eta$  every lr_step epochs.
  shuffle train indices; for each minibatch do
    load clips with augmentation; stack to  $[N, T, C, H, W]$ .
     $z \leftarrow \text{VideoCNN}(X)$ ;  $\ell \leftarrow \text{CE}(z, y)$ ; backward; SGD.
    evaluate val (and test if a target accuracy is set).
    if score improved: save_checkpoint.
  Load best weights; report test accuracy.

```

Figure 2: Training loop implemented in `trainer.hpp`.

5.2 Cache and model clip

To avoid re-decoding MPEG on every epoch, the loader first extracts `cache_frames=16` uniformly sampled frames and writes a HARF002 blob. `sample_model_clip` then reduces 16 frames to the model length $T = 8$:

- **Eval / no augment:** uniform subsample $t_i = i(T_{\text{in}} - 1)/(T_{\text{out}} - 1)$.
- **Train:** a random contiguous crop of 8 frames out of 16, optional horizontal flip (probability 1/2), and optional time reversal (probability 0.3).

If a clip is shorter than T , the last frame is repeated (`pad_clip`).

5.3 Prefetch

`train_one_epoch` optionally launches `std::async` to decode the next host batch while the current batch is on the device. `-workers 0` disables the extra thread. The first batch of an epoch prints mean and standard deviation of the stacked tensor; a near-zero warning catches a broken decoder.

6 Training procedure

Figure 2 is the loop in `train_video_cnn`. Each epoch shuffles training indices, applies augmentation, computes cross-entropy, back-propagates through VideoCNN, and steps SGD. Validation (and, if `-target-acc` is set, the test split) is evaluated without dropout or augmentation. The checkpoint with the best monitored score is written to `models/<dataset>_videocnn.harw` and reloaded for the final test printout.

Default hyperparameters (Table ??) are the values baked into `main.cpp` for UCF11. Batch size is further clipped by

$$N_{\max} = \left\lfloor \frac{16 \cdot 16 \cdot 112 \cdot 112}{T \cdot H \cdot W} \right\rfloor,$$

so 64×8 may use 32, while 112×16 is forced down to 16.

Resume is class-count checked but does not restore optimiser state: it is a warm start of θ , not a bit-exact continuation. `eval` bypasses training entirely, rebuilds the net from checkpoint metadata, and prints overall plus per-class accuracy on a chosen split.

7 CUDA backend

When `HAR_HAS_CUDA` is defined and a device is present, `default_device()` returns CUDA unless `HAR_DEVICE=cpu` is set. `src/cuda_ops.cu` provides:

- elementwise add/sub/mul/div and scalar variants;
- GEMM and transpose used by Linear;
- Conv2D forward/backward, ReLU, Dropout, max-pool, GAP;
- BatchNorm2D with a small device scratch buffer;
- TemporalConv1D forward/backward.

Kernels use a 1-D grid with 256 threads. Attention pooling currently stays on the host (it is $O(NTC)$ with $C = 256$, $T = 8$, and is not the bottleneck compared with the last 3×3 convolution).

Unified Memory simplifies the C++ API—the same `data()` pointer is valid on both sides—but it requires the activation cap of Section 7 and a matching driver/toolkit pair. The CMake comment documents a real failure mode: PTX produced by CUDA 13.3 cannot JIT on a 13.2 driver, which is why the project ships `86-real` cubin only.

The inference experiment in Section 9 ran on the CPU path (OpenCV decoder, no `nvcc` in the MSYS2 build). Numerical results therefore do not depend on GPU reductions.

8 Experiments

8.1 Setup

All numbers below use `models/ucf11_videocnnBest.harw`. Peeking the header yields `image_size=64`, `max_frames=8`, `in_channels=3`, `num_classes=11`, and the class list of Table ???. The tensor inventory (29 entries) matches current `VideoCNN::state_tensors()`. Inference ran with the rebuilt `har_cnn` (C++23, OpenCV 4.13, CPU).

8.2 Folder test on 10 random UCF11 clips

Ten videos were drawn uniformly from the 1600-clip tree, copied into `checkAcc/<class>/`, and classified with `har_cnn predict -top 3`. Folder names supply the ground-truth label. Table ??? reports every clip.

The model is clearly usable: six clips exceed 95% confidence on the correct class, including both diving examples and both tennis swings. The two errors are informative rather than random. `v_shooting_17_01` is basketball but the softmax put biking first (58%) and basketball third (14%). `v_walk_dog_10_02` is walking; biking won (57%) while walking remained a close second (40%). A third walking clip was correct but only by a thin margin versus horse-riding (38% vs. 36%). These confusions are consistent with YouTube HAR: handheld cameras, similar outdoor backgrounds, and cyclic leg motion shared by walking, biking, and riding.

8.3 How to read the best checkpoint

A sibling file `ucf11_videocnn.best86.harw` records a training run that reached about 86% on the monitored split. The `Best` file used here is the 256-channel architecture (2.36 MB), not the older 0.58 MB 64×8 snapshot. Because UCF11’s test split is a random 10% hold-out, a full `eval -split test` on ≈ 160 clips is the proper unbiased number; the 10-clip probe is a smoke test of the released weights, not a substitute for that split.

8.4 Smoke test and software correctness

`har_cnn smoke` trains a 4-filter Conv-ReLU-Pool-Linear net on 32 synthetic 16×16 images (centre-on vs. centre-off) for 80 epochs. Passing requires a strictly decreasing loss and at least 80% in-batch accuracy. This does not measure HAR quality, but it guards against silent breaks in convolution, pooling, SGD, or the CUDA/CPU switch.

8.5 Limitations

- **Resolution and length.** 64×8 discards fine racquet / ball detail and long temporal structure.
- **No optical flow.** Appearance-only features struggle when motion, not texture, is the cue.
- **UCF101** is supported but was not the focus of the checkpoint evaluated here; 101-way classification with the same 0.6M model is a harder regime.
- **Attention on CPU** and Unified Memory make the GPU path simpler than a production trainer (no cuDNN, no mixed precision).

9 Conclusions

Following the paradigm of Soft Computing for action recognition [1], AMSC-HAR shows that a course-scale C++ stack—tensors, layers, SGD, video I/O, checkpoints, and optional CUDA—is enough to train and deploy a real, efficient action classifier. This provides an excellent foundation for a distributed sensing agent in a Multi-Agent System (MAS). VideoCNN is small (5.9×10^5 parameters) and explicit: shared 2D features, residual temporal convolution, and attention pooling (Eq. (1)). On UCF11 the released best weights recognise most held-out YouTube clips with high confidence; a 10-video random probe scored 80%, with residual errors concentrated on walking / biking / basketball.

Natural extensions are a full test-split table with per-class metrics, optical-flow or two-stream fusion, a deeper temporal module, mixed-precision CUDA, and a proper UCF101 run with the official split files. The important property of the project is that each of those upgrades would touch a few headers rather than a third-party graph compiler.

References

- [1] E. H. Houssein, M. A. Mahdy, M. Kayed, H. Ouyang, and W. M. Mohamed, “Integrating Soft Computing and Multi-Agent for Action Recognition: Basics, Challenging and Future Direc-

tions,” *Archives of Computational Methods in Engineering*, 2025.